

Deciphering cell-cell communication in single-cell transcriptomics data

Install the required libraries

```
In [ ]: # Download a shell script to add CRAN repositories to Ubuntu Jammy
download.file("https://github.com/eddelbuettel/r2u/raw/master/inst/scripts/add_c
          "add_cranapt_jammy.sh")

# Change file permissions to make it executable
Sys.chmod("add_cranapt_jammy.sh", "0755")

# Run the shell script
system("./add_cranapt_jammy.sh")

# Enable the binary package manager (bspm) to handle system and CRAN packages
bspm::enable()
options(bspm.version.check = FALSE)
```

We will create an R function to performs system calls

```
In [ ]: # Define a function to execute shell commands and print the output
shell_call <- function(command, ...) {
  result <- system(command, intern = TRUE, ...)
  cat(paste0(result, collapse = "\n"))
}
```

Install required libraries

```
In [ ]: # Install required R packages
install.packages("R.utils")

# Install Seurat Wrappers from GitHub (commented)
# remotes::install_github('satijalab/seurat-wrappers@d28512f804d5fe05e6d68900ca9

# Install BiocManager if not already installed
if (!require("BiocManager", quietly = TRUE))
  install.packages("BiocManager", quiet = T)

# Install Harmony and LIANA from GitHub
install.packages("harmony")
remotes::install_github('saezlab/liana', upgrade=F)
```

```
In [ ]: # Install the Seurat package for single-cell analysis
remotes::install_version("Seurat", version = "4.4.0")
remotes::install_version("SeuratObject", version = "4.1.4")
```

```
In [ ]: BiocManager::install(c("sparseMatrixStats", "DelayedMatrixStats"))
```

Introduction

LIANA (Ligand-Receptor Inference Analysis) offers a variety of statistical methods to infer ligand-receptor interactions from single-cell transcriptomic data, using prior knowledge. This notebook aims to demonstrate how to use LIANA in a basic way with our data of interest.

Steps for Using LIANA:

1. Data Preparation:

- Upload your single-cell transcriptomic data in the appropriate format.
- Make sure the data is normalized and annotated correctly.

2. LIANA Installation and Import:

- Install the LIANA package, if it is not already installed, using the command `pip install liana`.
- Import the package into your notebook with `import liana`.

3. LIANA Configuration:

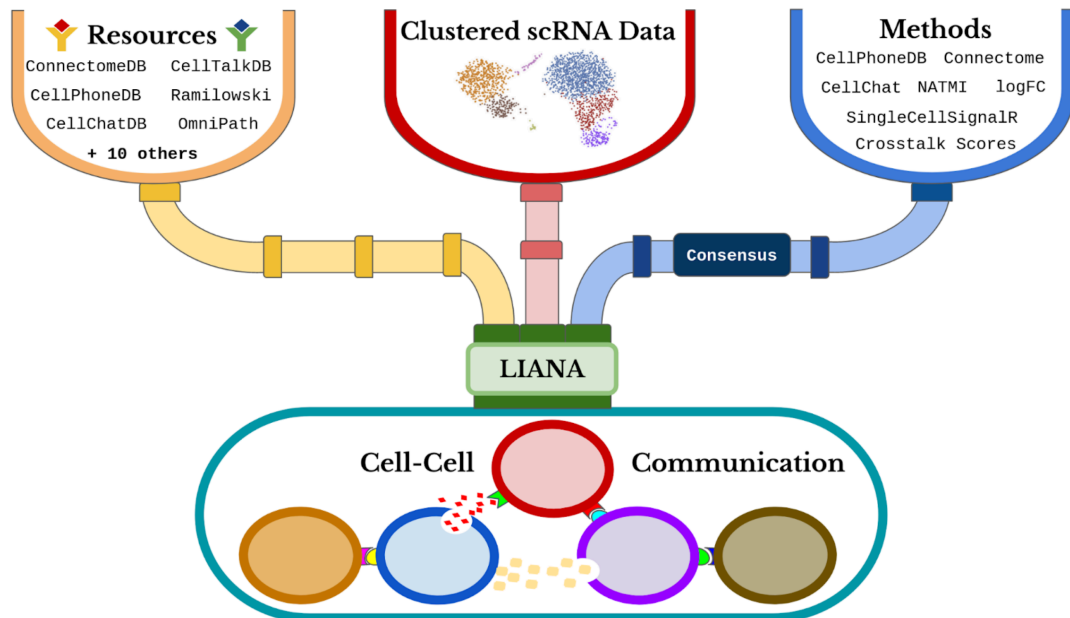
- Set the parameters required for the analysis, such as the cell type and signaling pathways of interest.
- Use LIANA-specific functions to define the ligand-receptor interactions you want to investigate.

4. Analysis Execution:

- Perform the analysis using the statistical methods available in LIANA.
- Analyze results to identify significant interactions between ligands and receptors in your samples.

5. Result Interpretation:

- Visualize results using built-in visualization tools or export data for further analysis.
- Interpret identified interactions in the biological context of your study.



```
In [ ]: # Load necessary libraries for data manipulation and analysis
library(tidyverse) # Collection of packages for data science
library(magrittr)  # Pipe operator
library(liana)     # Cell-cell communication analysis
library(Seurat)    # Single-cell RNA-seq analysis
```

```
In [ ]: sessionInfo()
```

In this section, we will showcase all the methods implemented by LIANA from various tools. Each method infers relevant ligand-receptor interactions based on different assumptions. Typically, each method returns two scores for each ligand-receptor pair:

1. **Magnitude (Strength) Score:** This score indicates the strength of the interaction.
2. **Specificity Score:** This score reflects how specific the interaction is to a given pair of cell identities.

```
In [ ]: # Show available methods in the current R session
show_methods()
```

The different resources of ligand-receptor interactions can be found here. The consensus integrates all the other resources.

```
In [ ]: # Show available resources in the current R session
show_resources()
```

Loading Data

Here we load our data of interest to study cell-cell communication.

```
In [15]: # Download the COVID-19 dataset (RDS file) from Dropbox
download.file("https://www.dropbox.com/sc/1fysew52kr8o2riahzubcw/BALF-COVID19")
```

```

In [16]: # Read the RDS file into a variable called 'testdata'
testdata <- readRDS('COVID.rds')

In [ ]: # Display an overview of the 'testdata' structure
testdata %>% dplyr::glimpse()

In [18]: # This specific condition 'group == "S" ' is being used to filter the rows. Only
testdata <- subset(x = testdata, subset = group == "S")

In [19]: # This function is used to get or set the identifiers of an object. The new colu
Idents(testdata) <- "celltype"

In [20]: # Normalize the single-cell RNA-seq data using Seurat
testdata <- Seurat::NormalizeData(testdata, verbose = FALSE)

In [ ]: # Display the structure of the processed dataset
testdata %>% dplyr::glimpse()

```

Running LIANA

To run LIANA, you can choose from any of the methods it supports. In this example, we will use the [CellPhoneDB](#) implementation.

The `liana_wrap` function invokes multiple methods, each operating with the provided resource(s). If no specific method is specified, `liana_wrap` will execute all methods implemented in LIANA. Additionally, the `consensus` resource is used by default.

```

In [ ]: cpdb_result <- liana_wrap(testdata, # This function of the LIANA package is used
                                     method = 'cellphonedb',
                                     resource = c('CellPhoneDB'), # Specifies that the Cell
                                     permutation.params = list(nperms=100, # Defines the pe
                                                                parallelize=FALSE, # Indicat
                                                                workers=4), # Number of work
                                     expr_prop=0.05) # Minimum proportion of expression to

In [ ]: # Display the structure of the CellPhoneDB interaction results
dplyr::glimpse(cpdb_result)

```

To run LIANA using multiple methods simultaneously, you can specify the desired methods in the `method` parameter. In this example, we will use CellPhoneDB, NATMI, SingleCellSignalR (sca), and the logFC approach.

```

In [ ]: # Run a more complex interaction analysis using multiple methods
complex_test <- liana_wrap(testdata,
                           method = c('cellphonedb', 'natmi', 'sca', 'logfc'),
                           resource = c('CellPhoneDB')) # Use CellPhoneDB resou

In [ ]: # Display the structure of the complex interaction results
dplyr::glimpse(complex_test)

```

One of the key features of LIANA is that it can compute a consensus ranking of the prediction of all methods employed to analyze cell-cell communication. By using the

function `liana_aggregate()` we can use all the results from every method used in the previous step.

```
In [ ]: # Aggregate interaction results across multiple methods
liana_consensus <- complex_test %>% liana_aggregate()
```

```
In [ ]: # Display the structure of the aggregated interaction results
dplyr::glimpse(liana_consensus)
```

Visualization and Interpretation

Dotplots can be generated to easily interpret important ligand-receptor pairs used by sender-receiver cell pairs.

Here, we preprocess the results of CellPhoneDB, then plot them. A filter to use only significant cases is applied (P-value < 0.05).

```
In [ ]: cpdb_int <- cpdb_result %>%
  # Only keep interactions with p-val <= 0.05
  filter(pvalue <= 0.05) %>% # This reflects interactions `specificity`
  rank_method(method_name = "cellphonedb", mode = "magnitude") %>% # Then rank a
  distinct_at(c("ligand.complex", "receptor.complex")) %>% # Keep top 20 interac
  head(20)
```

```
In [ ]: # Options(repr.plot.height = 12, repr.plot.width = 9)
# Plot cell-cell interactions using LIANA's dotplot function
scPlot <- cpdb_result %>%
  inner_join(cpdb_int, # Keep only the interactions of interest
    by = c("ligand.complex", "receptor.complex")) %>% # Invert
  mutate(pvalue = -log10(pvalue + 1e-10)) %>% # Transforms the p-value b
  liana_dotplot(source_groups = c("Epithelial"), # Creates a dot plot fo
    target_groups = c("Macrophages", "NK", "B", "T", "Neutro
    specificity = "pvalue", # Uses the p-value for dot size
    magnitude = "lr.mean", # Uses the mean log ratio for the
    show_complex = TRUE,
    size.label = "-log10(p-value)") + theme(axis.text.x = el

scPlot
# Save the plot as an image file
ggsave("01-liana_dotplot.png", plot = scPlot, bg = "white", dpi = 600, width = 1
```

Similarly, we can explore the consensus results.

```
In [ ]: # Options(repr.plot.height = 12, repr.plot.width = 9)
# Plot top 20 interactions from aggregated LIANA results
scPlot <- liana_consensus %>%
  liana_dotplot(source_groups = c("Macrophages"),
    target_groups = c("Macrophages", "NK", "B", "T", "Neutro
    ntop = 20) + theme(axis.text.x = element_text(angle = 90

scPlot
# Save the plot
ggsave("02-liana_dotplot.png", plot = scPlot, bg = "white", dpi = 600, width = 1
```

Overall potential of cells to communicate can be computed. Here, we can count the number of significant/important interactions. Then, they can be visualized through a

heatmap.

Similarly, we can compare the CellPhoneDB vs consensus results.

```
In [ ]: # Filter interactions with p-value ≤ 0.05 and plot frequency heatmap
liana_trunc <- cpdb_result %>% filter(pvalue <= 0.05)

# Generate and save heatmap
# png("03-heat_freq.png", bg = "white")
heat_freq(liana_trunc)
# dev.off()
```

```
In [ ]: # Filter consensus interactions with aggregate rank ≤ 0.01 and plot heatmap
liana_trunc <- liana_consensus %>% filter(aggregate_rank <= 0.01)

# Generate and save heatmap
# png("04-heat_freq.png", bg = "white")
heat_freq(liana_trunc)
# dev.off()
```

Extra Questions:

- How different are the results between two methods of your choice?
- Why can this happen?

```
In [35]: # List all functions available in the LIANA package
ls("package:liana")
```

'assign_lr_weights' · 'calculate_gini' · 'call_cellchat' · 'call_connectome' · 'call_italk' ·
'call_natmi' · 'call_sca' · 'call_squidpy' · 'cellchat_formatDB' · 'chord_freq' · 'compile_ligrec' ·
'conn_formatDB' · 'decomplexify' · 'decompose_tensor' · 'filter_nonabundant_celltypes' ·
'FormatConnectome' · 'FormatiTALK' · 'FormatSCA' · 'generate_homologs' ·
'generate_lr_geneset' · 'generate_omni' · 'generate_orthologs' · 'get_abundance_summary' ·
'get_c2c_factors' · 'get_connectome' · 'get_curated_omni' · 'get_logfc' · 'get_lr_resources' ·
'get_natmi' · 'get_sca' · 'heat_freq' · 'italk_formatDB' · 'join_means' · 'liana_aggregate' ·
'liana_bysample' · 'liana_call' · 'liana_defaults' · 'liana_dotplot' · 'liana_heatmap' · 'liana_pipe' ·
'liana_prep' · 'liana_tensor_c2c' · 'liana_wrap' · 'mean0' · 'minmax' ·
'plot_abundance_summary' · 'plot_c2c_cells' · 'plot_c2c_overview' · 'plot_context_boxplot' ·
'plot_context_heat' · 'plot_lr_heatmap' · 'rank_aggregate' · 'rank_method' · 'recomplexify' ·
'sca_formatDB' · 'select_resource' · 'show_homologene' · 'show_methods' · 'show_resources'